

Prompts utilizados

Núcleo Cognitivo da Aurora Siger (NCAS)

Atividade Integradora · Fase 5 · Ciência da Computação · FIAP

Grupo 29

Matheus Fuchelberguer · RM571321

Carlos Eugenio Andrade · RM570285

Rodrigo Gomes Dias · RM569142

Catálogo: ncas_core/prompts.json (versão 1.0)

Total de prompts: 6

1. Como os prompts foram projetados

O NCAS não realiza chamadas a APIs de inteligência artificial. Os prompts são montados pelo sistema e as respostas são simuladas localmente, o que mantém a demonstração reproduzível e sem dependências externas. O que está sendo demonstrado é a estrutura dos prompts e a lógica que decide qual deles usar em cada situação.

Todos os prompts seguem os quatro elementos de projeto estudados: cada um declara a tarefa, fornece o contexto que o modelo não teria como adivinhar, apresenta exemplos quando a tarefa depende de convenções internas, e especifica o formato exato da resposta esperada.

Elemento	Função	Onde aparece no catálogo
Tarefa	O que o modelo deve fazer	campo tarefa
Contexto	Informação que não está na instrução	campo contexto
Exemplos	Como uma boa saída se parece	campo exemplos
Formato	Forma, tom e extensão da resposta	campo formato_saida

1.1 Armazenamento e montagem

Os prompts não estão embutidos no código: ficam em `ncas_core/prompts.json`, junto do template que define a ordem dos elementos e da lista de variáveis que o sistema precisa preencher. Isso permite revisar ou acrescentar um prompt sem alterar a lógica da aplicação.

```
# ncas_core/prompts.py

def renderizar(self, id_prompt: str, **variaveis: str) -> str:
    prompt = self.obter(id_prompt)
    valores = {
        "contexto": prompt["contexto"],
        "tarefa": prompt["tarefa"],
        "formato_saida": prompt["formato_saida"],
        "exemplos": "\n".join(prompt.get("exemplos", [])),
    }
    for nome in prompt.get("variaveis", []):
        valores[nome] = variaveis.get(nome, f"<{nome} nao informado>")
    return prompt["template"].format(**valores)
```

Quando uma variável obrigatória não é informada, o sistema insere um marcador explícito em vez de enviar um prompt incompleto — assim a falha fica visível para o operador.

1.2 Técnicas utilizadas

Técnica	Quando foi escolhida	Prompts
zero-shot	Tarefa geral, que o modelo já domina	2 prompts
few-shot	Convenções internas da colônia	2 prompts
structured output	Saída consumida por outro programa	1 prompt

2. Catálogo de prompts

2.1 Resumir um alerta operacional

Identificador	Técnica	Objetivo
resumir_alerta	zero-shot	Transformar um alerta técnico extenso em um resumo de uma frase para o operador de plantão.

Prompt montado pelo sistema:

Você é o assistente do Núcleo Cognitivo da Aurora Siger, uma colônia marciana com 10 módulos e prioridades de P1 (essencial) a P5 (pesquisa).

Tarefa: Resuma o alerta operacional abaixo em uma única frase, começando pelo módulo afetado.

Alerta: LSS (Sistema de Suporte de Vida, P1) em estado DESLIGADO. Impacto em AGR, HAB e MED. 759 kW comprometidos.

Formato da resposta: Uma frase, no máximo 25 palavras, sem jargão técnico.

Resposta simulada:

LSS: falha no suporte de vida com impacto em habitação, medicina e agricultura; contingência acionada.

Por que o prompt foi escrito assim:

A tarefa é simples e bem definida, então não há necessidade de exemplos: o modelo já sabe resumir texto a partir do pré-treinamento. O que ele não tem como adivinhar é o domínio — por isso o contexto informa o que é a colônia e como funcionam as prioridades. O limite de 25 palavras evita que a resposta volte tão longa quanto o alerta original.

2.2 Classificar uma solicitação da tripulação

Identificador	Técnica	Objetivo
classificar_solicitacao	few-shot	Direcionar pedidos da tripulação para a fila correta usando exemplos de classificação já validados.

Prompt montado pelo sistema:

As categorias definem a fila de atendimento da colônia. EMERGENCIA interrompe a operação corrente; INFORMACAO não gera ordem de serviço.

Tarefa: Classifique a solicitação da tripulação em uma das categorias: EMERGENCIA, MANUTENCAO, SUPRIMENTO ou INFORMACAO.

Exemplos:

Solicitação: 'O nível de oxigênio do módulo HAB caiu abaixo do limite.' -> EMERGENCIA

Solicitação: 'A luminária da bancada 3 do laboratório queimou.' -> MANUTENCAO

Solicitação: 'Precisamos de mais filtros de água para o próximo ciclo.' -> SUPRIMENTO

Solicitação: 'Qual o consumo de energia previsto para amanhã?' -> INFORMACAO

Solicitação: A luminária da bancada 3 do laboratório queimou.

Formato da resposta: Apenas o nome da categoria, em letras maiúsculas, sem explicação.

Resposta simulada:

MANUTENCAO

Por que o prompt foi escrito assim:

Aqui o zero-shot falharia: as quatro categorias são convenções internas da colônia, e não conceitos gerais da língua. Os quatro exemplos mostram a fronteira entre elas — em especial a diferença entre um pedido que gera ordem de serviço (MANUTENCAO) e um que não gera (INFORMACAO). A instrução de responder apenas com a categoria evita texto extra que atrapalharia o roteamento automático.

2.3 Gerar resposta padronizada ao centro de controle

Identificador	Técnica	Objetivo
resposta_centro_controle	zero-shot	Padronizar a comunicação entre a colônia e o centro de controle na Terra.

Prompt montado pelo sistema:

O comunicado é transmitido com atraso de vários minutos e não pode gerar dúvida. Deve informar o que ocorreu, o impacto e a ação já tomada.

Tarefa: Escreva o comunicado oficial da ocorrência para o centro de controle.

Dados da ocorrência: LSS (Sistema de Suporte de Vida, P1) em estado DESLIGADO. Impacto em AGR, HAB e MED. 759 kW comprometidos.

Formato da resposta: Três linhas rotuladas: OCORRENCIA, IMPACTO e ACAO. Uma frase por linha.

Resposta simulada:

OCORRENCIA: Falha no módulo LSS (Sistema de Suporte de Vida), prioridade P1.

IMPACTO: Módulos AGR, HAB e MED dependem do LSS; 759 kW comprometidos.

ACAO: Protocolo de contingência acionado; revisão humana pendente.

Por que o prompt foi escrito assim:

O formato importa mais que o conteúdo: a comunicação com a Terra tem atraso de vários minutos, então o comunicado precisa responder de antemão as três perguntas óbvias. Os rótulos fixos OCORRENCIA, IMPACTO e ACAO garantem que nenhuma delas seja esquecida, sem precisar de exemplos.

2.4 Transformar registro técnico em linguagem simples

Identificador	Técnica	Objetivo
simplificar_registro	few-shot	Permitir que tripulantes sem formação técnica entendam o que aconteceu.

Prompt montado pelo sistema:

O leitor conhece a colônia, mas não conhece os códigos dos módulos nem as unidades de engenharia.

Tarefa: Reescreva o registro técnico em linguagem simples, como se explicasse a um colega de outra área.

Exemplos:

Registro: 'COM DEGRADADO, enlace redundante ativo.' -> 'A comunicação está instável, mas o canal reserva assumiu.'

Registro: 'PWR SHUTDOWN, 1030 kW indisponíveis.' -> 'A usina parou e a colônia está sem energia própria.'

Registro: LSS SHUTDOWN, 759 kW comprometidos.

Formato da resposta: Uma frase em linguagem cotidiana, sem siglas e sem números de engenharia.

Resposta simulada:

O sistema que mantém o ar e a água parou de funcionar e já acionamos o plano de emergência.

Por que o prompt foi escrito assim:

A dificuldade não é entender o registro, e sim calibrar o nível de linguagem. Os dois exemplos definem esse nível melhor do que qualquer descrição: mostram que siglas viram nomes completos e que valores de engenharia são substituídos por consequências práticas.

2.5 Gerar saída estruturada em JSON

Identificador	Técnica	Objetivo
diagnostico_estruturadostructured output		Produzir uma resposta que o próprio NCAS consiga gravar em dados_colonia.json sem tratamento manual.

Prompt montado pelo sistema:

A saída será lida por outro programa. Qualquer texto fora do objeto JSON quebra a integração.

Tarefa: Analise a ocorrência e retorne o diagnóstico no formato JSON especificado.

Ocorrência: LSS (Sistema de Suporte de Vida, P1) em estado DESLIGADO. Impacto em AGR, HAB e MED. 759 kW comprometidos.

Formato da resposta: Retorne APENAS um objeto JSON válido com os campos: classificacao (string: CRITICO, ATENCAO ou NORMAL), confianca (número entre 0 e 1), acao_recomendada (string) e origem (string).

Resposta simulada:

```
{
  "classificacao": "ATENCAO",
  "confianca": 0.92,
  "acao_recomendada": "Verificar o módulo de comunicação",
  "origem": "simulacao_NCAS"
}
```

Por que o prompt foi escrito assim:

A saída alimenta outro programa, então o prompt estabelece um contrato de dados: nomes dos campos, tipos esperados e valores permitidos em classificacao. A instrução de retornar APENAS o objeto JSON é essencial — sem ela o modelo tende a acrescentar uma frase introdutória que quebra o json.loads() do lado que consome a resposta.

2.6 Decidir a ordem de atendimento passo a passo

Identificador	Técnica	Objetivo
cadeia_raciocinio	chain-of-thought	Tornar auditável a decisão de qual ocorrência atender primeiro, expondo o raciocínio intermediário.

Prompt montado pelo sistema:

Há uma única equipe de manutenção disponível. A prioridade do módulo (P1 a P5) e o número de módulos dependentes definem a gravidade.

Tarefa: Decida qual ocorrência deve ser atendida primeiro. Pense passo a passo antes de concluir.

Ocorrências: A: falha no LSS (P1). B: falha no LOG (P4).

Formato da resposta: Liste os passos numerados do raciocínio e termine com uma linha iniciada por 'CONCLUSAO:'.

Resposta simulada:

```
1. A ocorrência A atinge o módulo LSS, de prioridade P1.
2. A ocorrência B atinge o módulo LOG, de prioridade P4.
3. O LSS possui três módulos dependentes; o LOG não possui nenhum.
4. A regra EMERGENCIA = FALHA . CRITICO é verdadeira apenas para a ocorrência A.
CONCLUSAO: atender primeiro a ocorrência A (LSS).
```

Por que o prompt foi escrito assim:

Priorizar entre ocorrências exige comparar prioridade e número de dependentes antes de concluir. Pedir a resposta direta costuma produzir uma escolha plausível mas sem justificativa auditável. Ao exigir os passos numerados, o operador consegue verificar cada etapa e discordar de um passo específico, e não apenas do resultado final.

3. Análise de melhoria da resposta

Um prompt raramente nasce pronto. Esta seção registra a iteração feita sobre o prompt de saída estruturada, que era o mais frágil do conjunto: sua resposta precisa ser lida por `json.loads()` e qualquer texto extra quebra a integração.

3.1 Primeira versão

```
Analise a ocorrência e responda em JSON com a classificação, a confiança e a ação recomendada.
```

Problemas observados na resposta simulada com essa formulação:

- O modelo antecedia o JSON com uma frase de apresentação.
- Os nomes dos campos variavam entre execuções (`classificacao` / `classe`).
- O campo de confiança ora vinha como número, ora como texto ("alta").
- Não havia lista fechada de valores para a classificação.

3.2 Versão adotada

```
Retorne APENAS um objeto JSON válido com os campos: classificacao (string: CRITICO, ATENCAO ou NORMAL), confianca (número entre 0 e 1), acao_recomendada (string) e origem (string).
```

Mudanças aplicadas e o efeito de cada uma:

Mudança	Problema que resolve
Nomear cada campo do objeto	Elimina a variação de nomenclatura
Declarar o tipo de cada campo	Garante número em confiança
Enumerar os valores permitidos	Fecha o domínio de <code>classificacao</code>
Exigir APENAS o objeto JSON	Remove o texto de apresentação
Explicar que outro programa lê	Justifica a rigidez do formato

Resultado: a resposta simulada passou a ser consumida diretamente por `json.loads()`, sem tratamento manual. O teste `test_saida_estruturada_declara_contrato_json` verifica exatamente isso — ele faz o parse da resposta e compara o campo `confianca`, falhando se o contrato for quebrado.

3.3 Princípio geral extraído da iteração

As quatro correções têm a mesma natureza: substituir uma expectativa implícita por uma instrução explícita. Sempre que a primeira versão de um prompt produziu resultado inconsistente, a causa foi

uma decisão que ficou a cargo do modelo quando deveria estar no prompt.

4. Uso dos prompts no sistema

O catálogo é carregado na inicialização do NCAS e fica disponível em duas situações distintas.

Onde	Comportamento
Menu 6 — Exibir prompts estruturados	Mostra o prompt montado e a resposta simulada de cada técnica
Menu 7 — Diagnosticar infraestrutura	Monta a recomendação com dados reais do módulo analisado

No diagnóstico, o prompt não é montado com texto fixo: ele recebe o módulo afetado, sua prioridade, o impacto calculado sobre as dependências, o protocolo recuperado da base local e o índice de risco estimado pelo modelo otimizado. É esse conjunto que caracteriza a etapa de recuperação de contexto antes da geração da resposta.

```
# ncas_core/aplicacao.py (trecho do diagnóstico)

documentos = self.base_conhecimento.buscar(consulta)
contexto = self.base_conhecimento.formatar_contexto(documentos)
resiliencia = avaliar_resiliencia(self.rede, modulo_id)
risco = otimizacao.prever_risco(self.modelo_risco, ...)

recomendacao = SimuladorIA.recomendacao_diagnostico(
    modulo.module_id, modulo.name, modulo.priority,
    str(modulo.status), impacto, contexto,
    str(resiliencia["nivel"]), texto_risco,
)
```

5. Limites e responsabilidade

Todas as respostas apresentadas neste documento são simuladas. Elas demonstram a estrutura dos prompts e o fluxo de decisão do sistema, não a inferência de um modelo de linguagem real. Nenhuma recomendação do NCAS deve ser aplicada sem revisão humana: todo diagnóstico é gravado com o campo `revisao_humana` em `PENDENTE` até que um operador o aprove ou rejeite.

Essa decisão de projeto responde a um risco concreto da IA generativa: uma resposta bem formatada e confiante pode estar errada. Manter a origem da resposta identificada, os dados do diagnóstico visíveis e a aprovação humana obrigatória reduz a chance de uma saída automática ser tratada como decisão final.